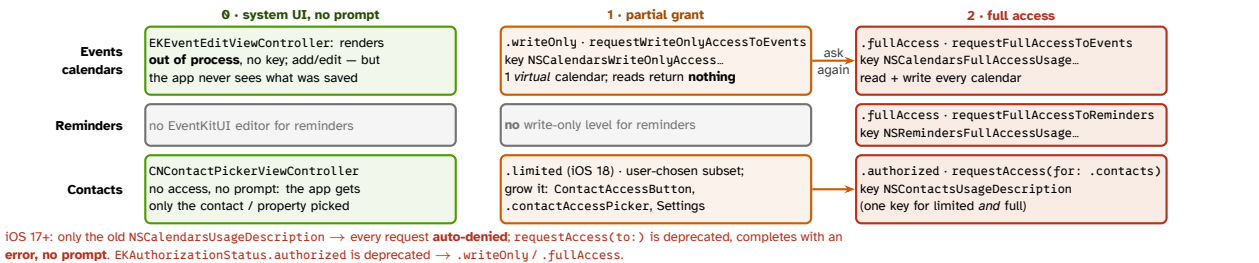


In one line: Calendars, reminders and contacts are **private databases** shared by every app. Climb the **access ladder** only as far as the feature needs: a **system UI** that needs **no permission** (EKEEventEditViewController, CNContactPickerViewController) → a **partial** grant (.writeOnly events, iOS 17 · .limited contacts, iOS 18) → **full access**. One long-lived EKEEventStore / CNContactStore; on “store changed”, refresh.

The access ladder — ask for the lowest rung that works



EventKit

- **One EKEEventStore**, created once and kept: expensive to initialise; objects from one store must not be used with another. Fetched *before* the grant? reset() after it.
- **Model:** EKSource (iCloud, Exchange, CalDAV, local) → EKCalendar (allowsContentModifications) → EKCalendarItem = EKEEvent | EKReminder.
- EKEEvent: startDate/endDate, isAllDay, timeZone (nil = floating, “lunch at noon” in any zone), recurrenceRules, alarms (EKAlarm(relativeOffset:), or a geofence: structuredLocation + proximity). EKReminder: dueDateComponents (DateComponents, not a Date), isCompleted.
- **Fetch:** predicateForEvents(withStart:end:calendars:) — span capped at **4 years**, silently — then events(matching:): **synchronous** (off main), **committed only, unsorted** (compareStartDate); a recurring event returns **one EKEEvent per occurrence**. Reminders: fetchReminders(matching:completion:) is **async**.
- **Save:** save(_:span:commit:), remove(_:span:commit:). Batch: commit: false × N, then one commit(); reset() drops the uncommitted.
- **Changes:** .EKEEventStoreChanged (any app, any sync) → **refresh the visible range**. One object you are editing: refresh() — false = deleted.

Example — write-only, weekly, with an alarm

```
let store = EKEEventStore() // app-wide, created once
func addSwimLessons(start: Date) async throws {
    guard try await store.requestWriteOnlyAccessToEvents() else { return }
    let e = EKEEvent(eventStore: store)
    e.title = "Swim lesson"
    e.calendar = store.defaultCalendarForNewEvents // user picks real one
    e.startDate = start; e.endDate = start + 45 * 60
    e.addRecurrenceRule(EKRecurrenceRule(recurrenceWith: .weekly,
        interval: 1, end: EKRecurrenceEnd(occurrenceCount: 10)))
    e.addAlarm(EKAlarm(relativeOffset: -15 * 60)) // 15 min before
    try store.save(e, span: .futureEvents, commit: true)
} // write-only: cannot read this event back afterwards
```

Identifiers — which one to store

eventIdentifier	local; changes if the event moves calendar; event(withIdentifier:) gives a series' first occurrence
calendarItem- Identifier	local, unique in this database
calendarItem- ExternalIdentifier	the server's id, same on every device; shared by all occurrences (+ start date); duplicates possible (ICS import, shared + invited)

Saving an edit to a recurring series

span: .thisEvent on the 15th → only it changes (a detached occurrence)



span: .futureEvents on the 15th → it and every later occurrence change



All six share one calendarItemExternalIdentifier; events(matching:) returns each one in range as its own EKEEvent, told apart by occurrenceDate.

Contacts

- CNContactStore: requestAccess(for: .contacts) — then branch on authorizationStatus(for:): a Bool cannot say limited vs full. Fetches do I/O: **off main**.
- **keysToFetch** gives *partial* contacts. Reading an unfetched key throws CNContactPropertyNotFetchedException — ObjC, a **crash** Swift cannot catch; check isKeyAvailable(_:). Display names need CNContactFormatter's required keys.
- **Unified:** one person linked across iCloud / Exchange / Gmail containers comes back as one CNContact (unifiedContacts(matching:keysToFetch:)). Big lists: identifiers first, details in batches.
- **Write:** mutableCopy() → CNMutableContact; CNSaveRequest add/update/delete; store.execute(req).
- **Sync:** CNContactStoreDidChange → refresh. Incremental: CNChangeHistoryFetchRequest (13) from a saved currentHistoryToken, run with enumerator(for:).
- **Limited** (18): fetches see only the subset. ContactAccessButton(queryString:) beside your search shows matches; a tap shares that contact. **Hidden** when .authorized. .contactAccessPicker(isPresented:) returns the **newly shared** ids.
- **Notes field:** com.apple.developer.contacts.notes needs Apple's **approval**; without it CNContactNoteKey → CLError.unauthorizedKeys.

Traps

- ! Full calendar access to “add to calendar” — the editor needs **no** access; write-only if you save silently.
- ! Write-only then fetching: empty results, *including your own events* — request full access first.
- ! Storing eventIdentifier as a sync key: moves with the calendar; use the external id + start date.
- ! Ten-year query: silently cut to **4 years** — page by range. Contacts: unfetched key = crash, not nil.

Remember

“UI, then partial, then full. One store, refresh on change.”

Events: *editor* → *write-only* → *full*; contacts: *picker* → *limited* → *authorized*; reminders: *full* or *nothing*.